

UNIVERSIDADE ESTADUAL DO MARANHÃO
PRÓ-REITORIA DE GRADUAÇÃO
CENTRO DE CIÊNCIAS TECNOLÓGICAS
NÚCLEO DE TECNOLOGIAS PARA EDUCAÇÃO
CURSO SUPERIOR DE TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO
DE SISTEMAS

ANDRÉ PEREIRA FERREIRA

**ANÁLISE COMPARATIVA DE DESEMPENHO NA ERA DA PERSISTÊNCIA EM
MEMÓRIA:** O Impacto da Arquitetura Worker do FrankenPHP no Framework
CodeIgniter 4 em Contraste com o Ecossistema Laravel.

ANDRÉ PEREIRA FERREIRA

**ANÁLISE COMPARATIVA DE DESEMPENHO NA ERA DA PERSISTÊNCIA EM
MEMÓRIA: O Impacto da Arquitetura Worker do FrankenPHP no Framework
CodeIgniter 4 em Contraste com o Ecossistema Laravel.**

Trabalho de Conclusão de Curso apresentado
ao Curso Superior de Tecnologia em Análise e
Desenvolvimento de Sistemas da Universidade
Estadual do Maranhão para obtenção do título
de Tecnólogo.

Orientadora: Prof. Fulano de Tal

ANDRÉ PEREIRA FERREIRA

**ANÁLISE COMPARATIVA DE DESEMPENHO NA ERA DA PERSISTÊNCIA EM
MEMÓRIA: O Impacto da Arquitetura Worker do FrankenPHP no Framework
CodeIgniter 4 em Contraste com o Ecossistema Laravel.**

Monografia apresentada ao Curso Superior de
Tecnologia em Análise e Desenvolvimento de
Sistemas da Universidade Estadual do
Maranhão para obtenção do título de
Tecnólogo.

Aprovado em: / / .

BANCA EXAMINADORA

Profa. Dra. Fulana da Silva (Orientadora)

Doutora em TI
Universidade Estadual do Maranhão

Prof. Dr. Paulo da Silva

Doutora em TI
Universidade Estadual do Maranhão

Profa. Dra. Amanda da Silva

Doutora em TI
Universidade Estadual do Maranhão

Ao Senhor meu Deus; à minha esposa, Jéssica;
e aos meus familiares, que depositam em mim
uma confiança maior do que julgo merecer.

AGRADECIMENTOS

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed ultrices metus est, a lobortis tellus semper a. Nam egestas tortor quis hendrerit maximus. Aenean eget nulla vitae ex porta condimentum id eu lacus. Mauris sit amet semper orci. Vivamus sed rhoncus velit, a dictum lacus. Nulla hendrerit, elit in mollis aliquam, justo turpis euismod elit, id rhoncus purus lectus sed enim. Nam et metus non augue ultrices luctus sed ac tellus. Mauris aliquam sem non volutpat faucibus. Lorem ipsum dolor sit amet, consectetur adipiscing elit.

Sed bibendum ex felis, at aliquet dui consectetur vitae. Curabitur blandit sem metus, vel blandit libero eleifend vitae. Ut placerat mauris vel odio luctus euismod ac in neque. Etiam sit amet rhoncus lorem. Vestibulum egestas rhoncus mi eget luctus. Etiam sollicitudin lacus est, vitae consequat nisi rhoncus nec. Mauris ligula mi, ultrices vel pulvinar sit amet, luctus eu ex.

Mauris in eros elementum, lobortis diam aliquam, sodales neque. Donec laoreet fringilla nunc ut dapibus. Duis felis mi, sagittis pulvinar risus eget, suscipit lacinia nisi. Quisque leo nibh, lacinia et faucibus sit amet, sollicitudin eget justo. Praesent eget tortor non ante malesuada consequat. Duis mollis lorem ut nisi placerat, vel fringilla dolor eleifend. Sed augue libero, sagittis vitae velit et, imperdiet rhoncus elit. Maecenas fermentum fringilla odio non auctor. Fusce et ligula turpis. Donec nec ornare metus. Mauris vitae est vitae odio aliquet vehicula eget blandit tortor.

RESUMO

Este trabalho analisa o impacto do uso de servidores de aplicação baseados em loops persistentes na performance do framework CodeIgniter 4 (CI4). O estudo compara o modelo tradicional PHP-FPM com o FrankenPHP operando em dois estados: o modo clássico e o modo Worker (não suportado nativamente pelo CI4). Utilizando o Laravel Octane como referência de mercado, a pesquisa avalia vazão (RPS) e latência sob estresse. A metodologia consiste em testes controlados via contêineres Docker e injeção de carga com Grafana K6 e Prometheus. Espera-se validar se a arquitetura persistente justifica a complexidade de implementação em frameworks legados ou aplicações monolíticas que utilizam CI4 no backend.

Palavras-chave: CodeIgniter 4; FrankenPHP; Desempenho; Benchmarking; PHP assíncrono.

ABSTRACT

This work analyzes the impact of using persistent loop-based application servers on the performance of the CodeIgniter 4 (CI4) framework. The study compares the traditional PHP-FPM model with FrankenPHP operating in two states: classic mode and Worker mode (not natively supported by CI4). Using Laravel Octane as a market benchmark, the research evaluates throughput (RPS) and latency under stress. The methodology consists of controlled tests via Docker containers and load injection with Grafana K6 and Prometheus. It is expected to validate whether the persistent architecture justifies the implementation complexity in legacy frameworks or monolithic applications that use CI4 in the backend.

Keywords: *CodeIgniter 4; FrankenPHP; Performance; Benchmarking; Asynchronous PHP.*

LISTA DE FIGURAS

Figura 1. Grau de envolvimento do usuário.....	9
--	---

LISTA DE TABELAS

Tabela 1. Relação entre usabilidade e qualidade afetiva.....	10
--	----

LISTA DE ABREVIATURAS E SIGLAS

CI4	Codeigniter 4
PHP-FPM	Servidor web tradicional (FastCGI Process Manager)
RPS	Requisições por Segundo (Requests Per Second)
UE	<i>User Experience</i>

SUMÁRIO

1. INTRODUÇÃO

A evolução constante das tecnologias web tem exigido que sistemas modernos ofereçam alta escalabilidade e baixo tempo de resposta, forçando desenvolvedores a tomarem decisões estratégicas sobre a arquitetura de execução. Historicamente, a linguagem PHP dominou o cenário do desenvolvimento server-side operando no modelo Request-Response-Die, onde o interpretador é iniciado, o script executado e todos os recursos destruídos ao final de cada requisição HTTP. Embora o PHP-FPM tenha trazido avanços ao manter processos persistentes, ele ainda opera de forma sequencial e bloqueante, o que gera um overhead de bootstrapping (inicialização do framework) que se repete a cada acesso, limitando o desempenho em cenários de alta carga.

A justificativa para a realização deste benchmark reside na necessidade de validar a transição deste modelo tradicional para o paradigma de memória persistente através de Application Servers modernos. O surgimento de tecnologias como o Swoole e o FrankenPHP permitiu que o PHP funcionasse com loops de eventos persistentes, mantendo a aplicação carregada na memória RAM e reduzindo drasticamente o tempo de inicialização. O ecossistema Laravel, através do Laravel Octane, já consolidou essa prática como um padrão de mercado para alta performance, lidando nativamente com a limpeza de estado e injeção de dependências.

Entretanto, o framework CodeIgniter 4 (CI4), embora amplamente utilizado pela sua leveza e simplicidade, não oferece suporte nativo ao modo Worker. Até é possível utilizar o modo clássico, mas não possui o mesmo desempenho. Para a comunidade de desenvolvimento web, é vital compreender se o esforço técnico de implementar soluções customizadas (como bridges de software) para habilitar o modo Worker no CI4 justifica os ganhos de vazão (RPS) e latência, ou se o uso do FrankenPHP em seu modo clássico já seria suficiente para superar o legado do PHP-FPM.

A relevância deste estudo para as comunidades de tecnologia é multifacetada:

Eficiência de Recursos: Em um cenário de infraestrutura em nuvem, onde o consumo de CPU e RAM impacta diretamente os custos operacionais, validar arquiteturas que otimizam o uso do hardware é uma prioridade estratégica.

Modernização de Sistemas Legados: Muitos sistemas baseados em CodeIgniter precisam escalar sem a necessidade de uma reescrita completa para outros frameworks ou linguagens como Go ou Node.js.

Adoção de Novas Ferramentas: O FrankenPHP é uma tecnologia de vanguarda que integra a libphp via threads em Go, e este benchmark provê dados empíricos sobre sua estabilidade e performance em diferentes modos de operação.

Portanto, este trabalho propõe-se a preencher uma lacuna de dados técnicos ao comparar o Cenário Clássico (FPM), o Cenário Moderno Estático (FrankenPHP Clássico) e o Cenário de Vanguarda (Modo Worker), utilizando o Laravel Octane como balizador de eficiência. Os resultados visam oferecer um guia prático para desenvolvedores que buscam estender a vida útil e a performance de aplicações PHP em ambientes de produção de alta demanda.

2. FUNDAMENTAÇÃO TEÓRICA

Para a fundamentação teórica das práticas aplicadas nesta produção acadêmica optou-se por utilizar artigos de similar interesse, mas principalmente a documentação das tecnologias em destaque que são objetos desta pesquisa.

3. METODOLOGIA

A presente pesquisa caracteriza-se como um estudo experimental e quantitativo, de natureza comparativa, focado na análise de desempenho de diferentes arquiteturas de execução para o framework CodeIgniter 4.

3.1. Isolamento do Ambiente Experimental

Para garantir a reprodutibilidade e evitar a interferência de processos externos, o experimento será conduzido em um ambiente virtualizado utilizando contêineres Docker. O uso de contêineres permite a imposição de limites estritos de hardware (CPU e memória RAM) via `deploy.resources.limits`, garantindo que cada tecnologia testada opere sob as mesmas condições de recursos.

O ecossistema de teste será composto por:

- **Cenário A (Legado):** CodeIgniter 4 sob PHP-FPM Alpine (1.0 CPU / 512MB RAM) e servidor Nginx.
- **Cenário B (Moderno Estático):** CodeIgniter 4 sob FrankenPHP em Modo Clássico, funcionando como substituto direto do PHP-FPM.

- **Cenário C (Vanguarda):** CodeIgniter 4 sob FrankenPHP em Modo Worker, utilizando uma bridge customizada para manter a aplicação em memória persistente.

3.2. Instrumentação e Ferramentas

A injeção de carga será realizada pela ferramenta Grafana k6, executada em um contêiner isolado (2.0 CPU / 512MB RAM) para que o processamento do injetor não compita com o servidor alvo. A coleta de dados de consumo de recursos (CPU e RAM) será feita através do subsistema de métricas do Docker e exportada para análise.

4. COMPARAÇÃO DOS RESULTADOS